Java OOP and Multithreading Viva Questions with Answers

Introduction to OOP

1. **What is Object-Oriented Programming? Explain its principles.**

    - OOP is a programming paradigm based on the concept of objects.
    - **Principles:**
        - **Encapsulation:** Bundling data and methods.
        - **Inheritance:** Reusing code through parent-child relationships.
        - **Polymorphism:** Same interface, different behaviors.
        - **Abstraction:** Hiding implementation details.

2. **What is the difference between a class and an object?**

    - A **class** is a blueprint for creating objects.
    - An **object** is an instance of a class containing data and behavior.

3. **Can you explain encapsulation with an example?**

Encapsulation hides internal details by using private variables and public getter/setter methods.
Example:

```java
class Person {
    private String name;
    public String getName() {
      return name;
    }
    public void setName(String name) {
      this.name = name;
    }
}
```

4. **What is inheritance? What are the types of inheritance supported in Java?**

    - Inheritance allows a class to inherit fields and methods from another class.
    - **Types:** Single, Multilevel, Hierarchical (no multiple inheritance with classes).

5. **What is polymorphism? How is it implemented in Java?**

    - Polymorphism allows one interface to represent multiple forms.
        - **Compile-time polymorphism:** Method overloading.
        - **Runtime polymorphism:** Method overriding.

6. **What is the difference between method overloading and method overriding?**

    - **Overloading:** Same method name, different parameter lists, same class.
    - **Overriding:** Same method signature, different implementation in subclass.

7. **What is abstraction, and how is it implemented in Java?**

    - Abstraction hides implementation details, exposing only functionality.
    - Implemented using **abstract classes** and **interfaces**.

Classes and Objects

9. **How do you create an object in Java?**

Using the `new` keyword:

```
ClassName obj = new ClassName();
```

10. **What is a constructor? What are the types of constructors in Java?**

    - A constructor initializes an object.
    - **Types:**
        - Default constructor.
        - Parameterized constructor.

11. **What is the difference between a default and a parameterized constructor?**

    - **Default constructor:** No parameters.
    - **Parameterized constructor:** Accepts arguments.

12. **What is the purpose of the `this` keyword in Java?**

    - Refers to the current object instance, resolves conflicts between class fields and parameters.

Inheritance and Interfaces

13. **What is the `super` keyword, and how is it used?**

    - Refers to the parent class.
        - Call parent methods: `super.methodName()`.
        - Call parent constructor: `super()`.

14. **What is the difference between `extends` and `implements`?**

    - **`extends`**: For class inheritance.
    - **`implements`**: For implementing interfaces.

15. **What are abstract classes and interfaces? How do they differ?**

    - **Abstract class:** Can have abstract and concrete methods.
    - **Interface:** By default, all methods are abstract (default and static methods allowed from Java 8+).

16. **Can a class implement multiple interfaces?**

    - Yes, because interfaces don't create ambiguity as they only define method signatures.

Polymorphism

17. **What is dynamic method dispatch in Java?**

    - Process where a call to an overridden method is resolved at runtime based on the object's type.

18. **What are the rules for method overriding?**

    - Same name, return type, and parameter list.
    - Access modifier can't be more restrictive.
    - Must not be `static` or `final`.

19. **What is the `@Override` annotation?**

    - Ensures a method overrides a superclass method, catching errors at compile time.

20. **Can we override a static method in Java? Why or why not?**

    - No, static methods belong to the class, not instances, so overriding doesn't apply.

Encapsulation and Access Modifiers

21. **What are access modifiers in Java?/ What is the difference between access modifiers?**

    - **Public:** Accessible everywhere.
    - **Private:** Accessible within the class.
    - **Protected:** Accessible within the package and subclasses.
    - **Default:** Package-private, accessible only within the package.

22. **How does Java achieve data hiding?**

    - Using `private` variables and public getter/setter methods.

Exception Handling

24. **What is the difference between `throw` and `throws`?**

    - `throw`: Used to explicitly throw an exception.
    - `throws`: Declares the exceptions a method may throw.

25. **What is the purpose of the `finally` block in exception handling?**

    - Ensures execution of code regardless of whether an exception occurs.

26. **Can a constructor throw an exception?**

    - Yes, by declaring exceptions with the `throws` keyword.

Advanced Topics

27. **What is the significance of the `final` keyword in Java?**

    - **Final variable:** Cannot be modified.
    - **Final method:** Cannot be overridden.
    - **Final class:** Cannot be extended.

28. **What are inner classes in Java?**

    - Classes defined inside another class.
    - **Types:** Static, Non-static, Anonymous.

29. **What is the difference between shallow and deep copy?**

    - **Shallow copy:** Copies references.
    - **Deep copy:** Copies actual objects.

30. **How does Java implement memory management?**

    - Uses **Garbage Collection** to remove unreferenced objects.

Miscellaneous

31. **What is the difference between `==` and `.equals()`?**

    - `==`: Compares references.
    - `.equals()`: Compares object values (can be overridden).

32. **What is the purpose of the `toString()` method?**

    - Provides a string representation of an object. Can be overridden.

33. **What is the use of the `instanceof` operator?**

    - Checks if an object is an instance of a specific class or subclass.

34. **Can we have a class inside an interface?**

    - Yes, it is implicitly static.

35. **What is the difference between composition and aggregation?**

    - **Composition:** "Part-of" relationship; objects cannot exist independently.
    - **Aggregation:** "Has-a" relationship; objects can exist independently.

Threads and Multithreading

36. **What is a thread in Java?**

    - A thread is the smallest unit of a process that runs independently.

37. **How do you create a thread in Java?**

Extending `Thread` class:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running.");
    }
}
MyThread t = new MyThread();
t.start();
```

Implementing `Runnable` interface:

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread running.");
    }
}
Thread t = new Thread(new MyRunnable());
t.start();
```

38. **What is the difference between `start()` and `run()` in threads?**

- **`start()`**********:** Starts a new thread and calls `run()` in a separate call stack.
- **`run()`**********:** Executes in the current thread.

39. **What is multithreading?**

- A feature that allows multiple threads to execute simultaneously, improving performance.

40. **What is synchronization in Java?**

Ensures thread safety by allowing only one thread to access critical code at a time.

```
synchronized void method() {
    // Critical section
}
```

41. **What is a deadlock in multithreading?**

- A situation where two or more threads are waiting for each other to release resources, causing a block.

42. **What is the difference between `wait()` and `sleep()`?**

- **`wait()`**********:** Releases the monitor lock and pauses until notified.
- **`sleep()`**********:** Pauses execution without releasing the lock.

43. **What is a thread pool in Java?**

- A collection of threads managed by a framework (e.g., `ExecutorService`) to reuse threads for tasks.